Knowledge Graph Manager

Documentation Technique

Application de bureau pour la gestion, la visualisation
et le raisonnement sur des graphes RDF et des ontologies OWL

Construit avec Python, PyQt6, rdflib, owlready2, SpaCy et Google Gemini

Table des matières

1 Vue d'ensemble du Projet **3**

2 Architecture du Projet **3**

3 Description Détaillée Fichier par Fichier **4**

3.1 Fichiers Racine . . . 4
3.1.1 `app.py` — Point d'entrée . . . 4
3.1.2 `requirements.txt` — Dépendances de production . . . 4
3.1.3 `requirements-dev.txt` — Dépendances de développement . . . 5
3.1.4 `.env` — Variables d'environnement . . . 5
3.1.5 `settings.xml` — Paramètres persistants . . . 5
3.1.6 `run.bat` — Lancement rapide . . . 5
3.1.7 `build_executable.bat` — Script de compilation . . . 5
3.1.8 `build_installer.iss` — Script Inno Setup . . . 6
3.1.9 `Knowledge Graph Manager.spec` — Configuration PyInstaller . . . 6
3.1.10 `example.ttl` — Fichier d'exemple . . . 6
3.1.11 `test_ontology.owl` / `test_ontology.rdf` — Ontologies de test . . . 6
3.2 Module `core/` — Logique Métier . . . 6
3.2.1 `core/rdf_manager.py` — Gestionnaire de graphe RDF . . . 6
3.2.2 `core/sparql_engine.py` — Moteur SPARQL . . . 6
3.2.3 `core/ontology_manager.py` — Gestionnaire d'ontologies . . . 7
3.2.4 `core/ontology_validator.py` — Validateur d'ontologies . . . 7
3.2.5 `core/reasoner.py` — Moteur de Raisonnement . . . 7
3.2.6 `core/knowledge_extractor.py` — Extracteur NLP . . . 7
3.2.7 `core/gemini_extractor.py` — Extracteur IA . . . 8
3.2.8 `core/settings_manager.py` — Gestionnaire de Paramètres . . . 8
3.3 Module `gui/` — Interface Graphique . . . 8
3.3.1 `gui/main_window.py` — Fenêtre Principale . . . 8
3.3.2 `gui/styles/dark_theme.qss` — Thème Sombre . . . 8
3.3.3 `gui/utils/syntax_highlighter.py` — Coloration Syntaxique . . . 9
3.3.4 `gui/utils/sparql_completer.py` — Auto-complétion . . . 9
3.3.5 `gui/widgets/graph_viewer.py` — Visualisation de Graphe . . . 9
3.3.6 `gui/widgets/query_widget.py` — Interface SPARQL . . . 9
3.3.7 `gui/widgets/text_kg_widget.py` — Conversion Texte → RDF . . . 10
3.3.8 `gui/widgets/ontology_tree.py` — Arbre Ontologique . . . 10
3.3.9 `gui/widgets/stats_widget.py` — Statistiques du Graphe . . . 10
3.3.10 `gui/widgets/namespace_dialog.py` — Gestion des Espaces de Noms 10
3.3.11 `gui/widgets/validation_dialog.py` — Résultats de Validation . . . 10
3.3.12 `gui/widgets/inferred_triples_dialog.py` — Triplets Inférés . . . 10
3.4 Répertoire `lib/` — Bibliothèques Embarquées . . . 11
3.5 Répertoire `data/` — Données . . . 11
3.6 Répertoire `scripts/` — Scripts de Débogage . . . 11
3.7 Répertoire `tests/` — Tests Unitaires . . . 11

4 Justification des Technologies **11**

4.1 Python . . . 12
4.2 PyQt6 . . . 12

4.3 rdflib 12
4.4 owlready2 12
4.5 PyVis + Vis.js 12
4.6 SpaCy 12
4.7 Google Gemini API 12
4.8 python-dotenv 13
4.9 PyInstaller + Inno Setup 13
4.10 XML pour les paramètres 13

5 Diagramme d'Architecture **13**

6 Flux de Données **14**

1 Vue d'ensemble du Projet

Knowledge Graph Manager est une application de bureau construite avec Python et PyQt6 permettant de :

- Charger, visualiser et manipuler des graphes RDF et des ontologies OWL.
- Interroger les données via SPARQL (SELECT, CONSTRUCT, ASK, DESCRIBE, INSERT, DELETE).
- Extraire automatiquement des triplets RDF à partir de texte libre grâce au NLP (SpaCy) et à l'IA générative (Google Gemini).
- Exécuter du raisonnement automatisé (HermiT / Pellet) pour inférer de nouveaux faits.
- Valider la cohérence logique d'une ontologie.
- Gérer les espaces de noms, exporter les résultats, et naviguer visuellement dans les classes et propriétés.

L'application peut être compilée en un exécutable Windows autonome via PyInstaller, et distribuée via un installateur Inno Setup.

2 Architecture du Projet

```
knowledge-Graph-tools/
|
|-- app.py                          # Point d'entree principal
|-- requirements.txt                # Dependances de production
|-- requirements-dev.txt            # Dependances de developpement
|-- .env                            # Cle API (Gemini)
|-- settings.xml                    # Parametres persistants
|-- run.bat                         # Script de lancement rapide
|-- build_executable.bat            # Script de compilation
   PyInstaller
|-- build_installer.iss             # Script Inno Setup (installateur
   Windows)
|-- Knowledge Graph Manager.spec    # Configuration PyInstaller
|-- icon.ico                        # Icone de l'application
|-- example.ttl                     # Fichier Turtle d'exemple
|-- test_ontology.owl               # Ontologie d'exemple pour tests
|-- test_ontology.rdf               # Meme ontologie en RDF/XML
|-- ontology.owl                    # Ontologie de travail principale
|
|-- core/                           # Logique metier (backend)
|   |-- rdf_manager.py              # Gestion du graphe RDF (rdflib)
|   |-- sparql_engine.py            # Moteur d'execution SPARQL
|   |-- ontology_manager.py         # Chargement et inspection d'
   ontologies
|   |-- ontology_validator.py       # Validation de coherence
|   |-- reasoner.py                 # Moteur de raisonnement (HermiT/
   Pellet)
|   |-- knowledge_extractor.py      # Extraction de triplets via SpaCy
    NLP
|   |-- gemini_extractor.py         # Extraction de triplets via
   Gemini API
|   `-- settings_manager.py         # Gestion des parametres (XML + .
   env)
```

```
|
|-- gui/                              # Interface graphique (frontend)
|   |-- main_window.py                # Fenetre principale
|   |-- styles/
|   |   `-- dark_theme.qss            # Feuille de style sombre (QSS)
|   |-- utils/
|   |   |-- syntax_highlighter.py     # Coloration syntaxique SPARQL
|   |   `-- sparql_completer.py       # Auto-completion SPARQL
|   `-- widgets/
|       |-- graph_viewer.py           # Visualisation de graphe (PyVis)
|       |-- query_widget.py           # Interface SPARQL
|       |-- text_kg_widget.py         # Conversion texte -> RDF
|       |-- ontology_tree.py          # Arbre hierarchique classes/
   proprietes
|       |-- stats_widget.py           # Statistiques du graphe
|       |-- namespace_dialog.py       # Gestion des prefixes
|       |-- validation_dialog.py      # Resultats de validation
|       `-- inferred_triples_dialog.py  # Triplets inferes
|
|-- lib/                              # Bibliotheques externes
   embarquees
|   |-- bindings/utils.js             # Utilitaires JS de PyVis
|   |-- tom-select/                   # Bibliotheque Tom Select
|   `-- vis-9.1.2/                    # Vis.js v9.1.2
|
|-- data/                             # Donnees de test et exemples
|-- scripts/                          # Scripts utilitaires de debogage
`-- tests/                            # Tests unitaires (pytest)
```

3 Description Détaillée Fichier par Fichier

3.1 Fichiers Racine

3.1.1 `app.py` — Point d'entrée

Le point d'entrée principal de l'application. Il initialise `QApplication` (PyQt6), instancie la `MainWindow`, et lance la boucle d'événements. Ce fichier est volontairement minimaliste pour séparer clairement le lancement de la logique applicative.

3.1.2 `requirements.txt` — Dépendances de production

Déclare toutes les bibliothèques Python nécessaires à l'exécution :

Dépendance	Version	Rôle
`PyQt6`	$\geq$6.5, <7	Framework GUI de bureau
`PyQt6-WebEngine`	$\geq$6.5, <7	Moteur de rendu web intégré (pour les graphes PyVis)
`rdflib`	$\geq$6.3	Manipulation de graphes RDF (parsing, sérialisation, SPARQL)
`owlready2`	$\geq$0.45	Chargement d'ontologies OWL, raisonnement (HermiT/Pellet)
`pyvis`	$\geq$0.3	Génération de visualisations de réseau interactives (HTML/Vis.js)

Dépendance	Version	Rôle
`networkx`	≥3.0	Dépendance de PyVis pour la structure de graphe
`spacy`	≥3.5	Pipeline NLP pour l'extraction de connaissances
`spacy-lookups-data`	—	Données de lookup pour SpaCy (lemmatisation, etc.)
`google-generativeai`	≥0.3	SDK de l'API Google Gemini
`python-dotenv`	≥1.0	Chargement de variables d'environnement depuis `.env`

3.1.3 `requirements-dev.txt` — Dépendances de développement

Outils nécessaires uniquement pour le développement et la compilation :

- `pyinstaller` ≥6.0 — Compilation en exécutable Windows.
- `pytest` ≥7.0 — Exécution des tests unitaires.

3.1.4 `.env` — Variables d'environnement

Stocke la clé API Google Gemini (`GEMINI_API_KEY`). Ce fichier est exclu du contrôle de version (`.gitignore`) pour des raisons de sécurité. Il est chargé automatiquement au démarrage via `python-dotenv`.

3.1.5 `settings.xml` — Paramètres persistants

Fichier XML qui persiste les préférences utilisateur :

- Modèle Gemini sélectionné
- Thème sombre activé/désactivé
- Profil de raisonnement (HermiT/Pellet)
- Historique des requêtes SPARQL

Ce fichier est également exclu du `.gitignore` car il contient des préférences locales.

3.1.6 `run.bat` — Lancement rapide

Script batch Windows qui détecte la présence d'un environnement virtuel (`.venv`) et lance l'application avec le bon interpréteur Python. Un double-clic suffit pour lancer l'application.

3.1.7 `build_executable.bat` — Script de compilation

Automatise la compilation avec PyInstaller. Il :

1. Installe `pyinstaller` dans le venv si absent.
2. Exécute PyInstaller avec les options nécessaires (`-windowed`, `-collect-all`, `-hidden-import`).
3. Intègre les styles QSS, les modèles SpaCy, et les fichiers internes de owlready2, pyvis et rdflib.

3.1.8 `build_installer.iss` — Script Inno Setup

Configuration de l'installateur Windows. Il :

- Copie le contenu de `dist/Knowledge Graph Manager/` vers `Program Files`.
- Crée des raccourcis menu démarrer et bureau.
- Permet le lancement optionnel après installation.
- Génère un fichier `Knowledge_Graph_Manager_Setup.exe` dans le dossier `Installateur/`.

3.1.9 `Knowledge Graph Manager.spec` — Configuration PyInstaller

Fichier de spécification généré par PyInstaller, personnalisé pour inclure :

- Les fichiers de données de SpaCy (`en_core_web_sm`), owlready2, pyvis et rdflib.
- Les imports cachés (`spacy`, `en_core_web_sm`).
- Les styles QSS de l'interface.
- L'icône personnalisée.

3.1.10 `example.ttl` — Fichier d'exemple

Fichier Turtle contenant un petit graphe de connaissances sur Apple Inc. (fondateurs, siège social), servant de démonstration pour les fonctionnalités de l'application.

3.1.11 `test_ontology.owl` / `test_ontology.rdf` — Ontologies de test

Fichiers ontologie en formats OWL et RDF/XML utilisés pour tester le chargement, la hiérarchie de classes, et le raisonnement.

3.2 Module `core/` — Logique Métier

3.2.1 `core/rdf_manager.py` — Gestionnaire de graphe RDF

Classe principale : `RDFManager`

Encapsule un objet `rdflib.Graph` et fournit des opérations de haut niveau :

- **`load_file(path)`** : Charge un fichier RDF en détectant automatiquement le format à partir de l'extension (`.ttl`, `.rdf`, `.owl`, `.nt`, `.jsonld`, `.trig`, `.nq`).
- **`save_file(path, fmt)`** : Sérialise le graphe dans le format spécifié.
- **`get_statistics()`** : Calcule et retourne des métriques (nombre de triplets, sujets, prédicats, objets, classes).
- **`get_namespaces()` / `bind_namespace()` / `remove_namespace()`** : Gestion des préfixes d'espaces de noms.

3.2.2 `core/sparql_engine.py` — Moteur SPARQL

Classe principale : `SPARQLEngine`

Exécute des requêtes SPARQL sur le graphe géré par `RDFManager` :

- **`execute_query(query_str)`** : Exécute SELECT/CONSTRUCT/ASK/DESCRIBE. Retourne les résultats et le temps d'exécution.
- **`execute_update(query_str)`** : Exécute INSERT/DELETE. Retourne un dictionnaire avec le delta de triplets.

— **`is_update_query(query_str)`** : Détecte si une requête est un UPDATE.
— **`format_results(results)`** : Convertit les résultats rdflib en dictionnaire structuré pour l'interface.

3.2.3 `core/ontology_manager.py` — Gestionnaire d'ontologies

Classe principale : `OntologyManager`

Utilise **owlready2** pour charger et inspecter des ontologies OWL/RDFS :

— **`load_ontology(path)`** : Charge une ontologie. Gère la conversion de chemins Windows en IRI valides.
— **`get_classes()` / `get_properties()`** : Liste les classes et propriétés de l'ontologie.
— **`get_hierarchy()`** : Construit un arbre hiérarchique récursif des classes (racine : `owl:Thing`).
— **`get_properties_info()`** : Retourne les métadonnées des propriétés (type, domaine, portée).

> **Justification de l'usage de owlready2 en parallèle de rdflib** : owlready2 fournit un accès orienté objet aux concepts OWL (classes, propriétés, hiérarchie, raisonnement) que rdflib ne peut pas offrir nativement. rdflib est utilisé pour la manipulation générale du graphe RDF et SPARQL.

3.2.4 `core/ontology_validator.py` — Validateur d'ontologies

Classe principale : `OntologyValidator`

Vérifie la cohérence logique d'une ontologie :

1. Vérifie la présence de classes, propriétés et individus.
2. Détecte les propriétés sans domaine ou portée définie.
3. Exécute le raisonneur HermiT pour trouver les classes insatisfiables.
4. Identifie les classes orphelines (sans parent autre que `Thing`).

Chaque résultat est classé par sévérité : `error`, `warning`, ou `info`.

3.2.5 `core/reasoner.py` — Moteur de Raisonnement

Classe principale : `ReasoningEngine`

Effectue du raisonnement automatisé sur les ontologies :

— **`run_reasoner(path, type)`** : Charge l'ontologie dans un `World` isolé, exécute le raisonneur choisi (HermiT ou Pellet via owlready2), sauvegarde le résultat dans un fichier temporaire, puis le parse avec rdflib.
— **`apply_inference_to_graph(original, inferred)`** : Calcule le différentiel entre le graphe original et le graphe après raisonnement.

3.2.6 `core/knowledge_extractor.py` — Extracteur NLP

Classe principale : `KnowledgeExtractor`

Utilise **SpaCy** (`en_core_web_sm`) pour extraire des triplets RDF à partir de texte :

— Analyse les dépendances syntaxiques pour identifier les relations sujet-prédicat-objet.

— Résout les pronoms via un mécanisme de coréférence simplifié.
— Expandit les conjonctions (« Alice et Bob » → deux sujets distincts).
— Ajoute des types OWL (`NamedIndividual`, `ObjectProperty`) et des types NER (`foaf:Person`, `foaf:Organization`, `schema:Place`).

3.2.7 `core/gemini_extractor.py` — Extracteur IA

Classe principale : `GeminiExtractor`

Utilise l'API **Google Gemini** pour extraire des triplets RDF :

— Envoie un prompt structuré demandant une sortie en format Turtle.
— Parse la réponse et la convertit en graphe rdflib.
— Gère les erreurs d'API (authentification, quota, réseau).
— Permet de lister dynamiquement les modèles disponibles.

3.2.8 `core/settings_manager.py` — Gestionnaire de Paramètres

Classe principale : `SettingsManager`

Gère la persistance des paramètres :

— **Clé API** : Chargée depuis `.env` (sécurité), avec fallback XML (compatibilité legacy).
— **Autres paramètres** : Stockés dans `settings.xml` (modèle Gemini, thème, profil de raisonnement, historique SPARQL).
— L'historique des requêtes est limité à 50 entrées.

3.3 Module `gui/` — Interface Graphique

3.3.1 `gui/main_window.py` — Fenêtre Principale

Classe principale : `MainWindow` (hérite de `QMainWindow`)

Orchestre l'ensemble de l'interface :

— **Barre de menus** : File (charger, exporter, réinitialiser), View (thème), Reasoning (activer/exécuter), Tools (namespaces, validation).
— **Barre d'outils** : Sélecteur de profil de raisonnement, bouton d'exécution.
— **Onglets centraux** : Visualization, SPARQL Query, Text to RDF.
— **Panneaux latéraux (Docks)** : Ontology Hierarchy (gauche), Graph Statistics (droite).
— **Signaux** : `graph_merged` et `graph_updated` propagent les changements de graphe.

3.3.2 `gui/styles/dark_theme.qss` — Thème Sombre

Feuille de style QSS (Qt Style Sheets) définissant le thème sombre de l'application. Couvre : couleurs de fond, texte, onglets, boutons, champs de saisie, tableaux, arbres, menus et barres de défilement.

3.3.3 `gui/utils/syntax_highlighter.py` — Coloration Syntaxique

Classe principale : `SPARQLHighlighter` (hérite de `QSyntaxHighlighter`)

Applique la coloration syntaxique dans l'éditeur SPARQL :

- **Bleu gras** : Mots-clés SPARQL (`SELECT`, `WHERE`, `INSERT`, etc.).
- **Vert** : Variables (`?var`, `$var`).
- **Violet** : Fonctions built-in (`STR`, `COUNT`, `REGEX`, etc.).
- **Cyan foncé** : URIs (`<http://...>`) et QNames (`prefix:suffix`).
- **Rouge foncé** : Chaînes de caractères.
- **Gris italique** : Commentaires (`# ...`).

3.3.4 `gui/utils/sparql_completer.py` — Auto-complétion

Classe principale : `SPARQLCompleter`

Fournit l'auto-complétion dans l'éditeur SPARQL via un popup `QListWidget` :

- Propose les mots-clés et fonctions SPARQL 1.1.
- Suggère les variables déjà utilisées dans la requête.
- Propose les préfixes communs après le mot-clé `PREFIX`.
- Propose les préfixes d'espaces de noms chargés dans le graphe.
- S'active après 2 caractères tapés, limité à 15 suggestions.

3.3.5 `gui/widgets/graph_viewer.py` — Visualisation de Graphe

Classes : `GraphViewer`, `GraphWebPage`

Affiche le graphe RDF comme un réseau interactif :

- Convertit le graphe rdflib en réseau PyVis.
- Rend le réseau dans un `QWebEngineView` via un fichier HTML temporaire.
- Supporte 3 modes de mise en page : Force Directed, Hierarchical, Barnes Hut.
- Interception des clics : sélection de nœuds, ouverture d'URIs HTTP dans le navigateur.
- Bouton « Remove Selected Node » pour supprimer un nœud et ses triplets associés.
- Limite d'affichage à 500 triplets pour les performances.

3.3.6 `gui/widgets/query_widget.py` — Interface SPARQL

Classe principale : `QueryWidget`

Interface complète pour les requêtes SPARQL :

- Éditeur avec coloration syntaxique et auto-complétion.
- Boutons : Execute Query (SELECT/ASK/CONSTRUCT/DESCRIBE) et Execute UPDATE (INSERT/DELETE).
- Résultats affichés dans deux onglets : Table et Visualization.
- Historique des requêtes avec chargement rapide par double-clic.
- Export des résultats en JSON, CSV ou XML.
- Détection automatique des requêtes UPDATE.

3.3.7 `gui/widgets/text_kg_widget.py` — Conversion Texte → RDF

Classe principale : `TextKGWidget`

Widget à deux onglets pour l'extraction de connaissances :

1. **Gemini AI** : Configuration de la clé API et du modèle, extraction via l'API, visualisation du résultat, fusion vers le graphe principal.
2. **SpaCy NLP** : Extraction locale sans API, même workflow de visualisation et fusion.

Fonctionnalités communes : tableau des triplets extraits, visualisation du sous-graphe généré, export RDF (Turtle, RDF/XML, N-Triples), et fusion vers le graphe principal.

3.3.8 `gui/widgets/ontology_tree.py` — Arbre Ontologique

Classe principale : `OntologyTree`

Affiche la structure de l'ontologie dans deux onglets :

- **Classes** : Arbre hiérarchique récursif (racine : `Thing`), avec nom et IRI.
- **Properties** : Groupées par type (Object Properties, Data Properties), avec domaine et portée.

3.3.9 `gui/widgets/stats_widget.py` — Statistiques du Graphe

Classe principale : `StatsWidget`

Panneau compact affichant en temps réel : nombre de triplets, sujets, prédicats, objets et classes dans le graphe.

3.3.10 `gui/widgets/namespace_dialog.py` — Gestion des Espaces de Noms

Classe principale : `NamespaceDialog`

Dialogue modal permettant de visualiser tous les préfixes d'espaces de noms, d'ajouter/-mettre à jour un binding, et de supprimer un binding existant.

3.3.11 `gui/widgets/validation_dialog.py` — Résultats de Validation

Classe principale : `ValidationDialog`

Affiche les résultats de la validation d'ontologie : résumé avec compteurs, tableau trié par sévérité (erreurs en premier), icônes visuelles ($\times$, $\triangle$, i) et codes couleur.

3.3.12 `gui/widgets/inferred_triples_dialog.py` — Triplets Inférés

Classe principale : `InferredTriplesDialog`

Affiche les résultats du raisonnement automatisé : résumé avec temps d'exécution, nombre de nouveaux triplets, tableau des triplets inférés (Subject, Predicate, Object), et bouton d'export.

3.4 Répertoire `lib/` — Bibliothèques Embarquées

- **`lib/bindings/utils.js`** : Utilitaires JavaScript pour la mise en surbrillance des nœuds et le filtrage dans les visualisations PyVis.
- **`lib/vis-9.1.2/`** : Bibliothèque Vis.js v9.1.2 embarquée pour le rendu des visualisations de réseau.
- **`lib/tom-select/`** : Bibliothèque Tom Select pour les sélecteurs enrichis dans les interfaces HTML générées par PyVis.

3.5 Répertoire `data/` — Données

- **`dev.ttl`** / **`test.ttl`** : Fichiers Turtle de développement et de test.
- **`examples/`** : Ontologies d'exemple (`pizza.owl`, `test.owl`, etc.).
- **`reasoners/`** : Données de test spécifiques pour le raisonnement (`famille_data.rdf`, `famille_complete.rdf`).

3.6 Répertoire `scripts/` — Scripts de Débogage

- **`debug_extraction.py`** : Teste l'extraction NLP. Exécute SpaCy sur un texte d'exemple et affiche les entités et triplets extraits.
- **`debug_reasoner.py`** : Teste le moteur de raisonnement avec une ontologie simple avec transitivité.
- **`verify_pyvis.py`** : Vérifie que PyVis fonctionne correctement (génération HTML, inclusion de Vis.js).

3.7 Répertoire `tests/` — Tests Unitaires

Le projet contient **14 fichiers de test** couvrant :

Fichier	Module testé
`test_rdf_manager.py`	Chargement, sauvegarde et statistiques RDF
`test_sparql_engine.py`	Exécution de requêtes SPARQL
`test_ontology_manager.py`	Chargement d'ontologies et hiérarchie
`test_reasoner.py`	Moteur de raisonnement
`test_knowledge_extractor.py`	Extraction NLP via SpaCy
`test_gemini_ui.py`	Interface Gemini AI
`test_graph_interaction.py`	Interactions avec le graphe (sélection, suppression)
`test_new_features.py`	Nouvelles fonctionnalités
`test_ontology_refresh.py`	Rafraîchissement de l'ontologie après modifications
`test_reasoning_ui.py`	Interface de raisonnement
`test_settings_persistence.py`	Persistance des paramètres
`test_namespace_manager.py`	Gestion des espaces de noms
`test_syntax_highlighter.py`	Coloration syntaxique SPARQL
`test_theme.py`	Thème sombre

4 Justification des Technologies

4.1 Python

Python a été choisi pour son écosystème riche pour le Web sémantique (rdflib, owlready2, SpaCy), sa syntaxe claire et productive pour un projet universitaire, sa large communauté et documentation abondante, et sa compatibilité avec les bibliothèques d'IA générative (`google-generativeai`).

4.2 PyQt6

PyQt6 a été préféré à Tkinter, Kivy ou une interface web pour plusieurs raisons :

- **Richesse des widgets** : QTreeWidget, QTableWidget, QDockWidget, QTabWidget — essentiels pour une interface complexe multi-panneaux.
- **QWebEngineView** : Permet d'embarquer un rendu HTML/JS complet (Vis.js) directement dans l'application de bureau, sans serveur web.
- **Thèmes via QSS** : Personnalisation visuelle avancée (thème sombre) sans dépendance externe.
- **Maturité** : Bindings Qt officiels, stabilité éprouvée, documentation exhaustive.
- **Cross-platform** : Fonctionne sur Windows, macOS et Linux.

4.3 rdflib

rdflib est le standard de facto en Python pour la manipulation de graphes RDF. Il supporte tous les formats requis (Turtle, RDF/XML, N-Triples, JSON-LD, TriG, N-Quads), dispose d'un moteur SPARQL 1.1 intégré, et gère nativement les espaces de noms.

4.4 owlready2

owlready2 est utilisé en complément de rdflib pour :

- **Modèle objet OWL** : Accès direct aux classes, propriétés, individus, et hiérarchie.
- **Raisonneur intégré** : Interface transparente avec HermiT (Java) et Pellet pour l'inférence automatique.
- **Validation** : Détection des classes insatisfiables et des incohérences logiques.

4.5 PyVis + Vis.js

PyVis génère des visualisations de réseau interactives en HTML/JS à partir de Python, basé sur Vis.js. Il supporte plusieurs algorithmes de layout et fonctionne hors-ligne dans QWebEngineView grâce au mode `cdn_resources="in_line"`.

4.6 SpaCy

SpaCy est utilisé pour l'extraction NLP grâce à son pipeline de niveau industriel avec modèles pré-entraînés (`en_core_web_sm`), son analyse des dépendances syntaxiques, sa reconnaissance d'entités nommées (NER) intégrée, et sa performance élevée en local.

4.7 Google Gemini API

Gemini complète SpaCy pour la **compréhension sémantique profonde** : un modèle de langage peut extraire des relations implicites et complexes que le parsing syntaxique ne capture pas, et produit directement du Turtle valide.

4.8 python-dotenv

python-dotenv sépare les secrets (clé API) du code et des paramètres, suivant la convention standard du fichier `.env`, compatible `.gitignore`.

4.9 PyInstaller + Inno Setup

Cette chaîne de distribution permet de :

— **PyInstaller** : Compiler l'application Python et toutes ses dépendances en un dossier autonome, sans nécessiter l'installation de Python chez l'utilisateur final.

— **Inno Setup** : Créer un installateur Windows professionnel (.exe) avec raccourcis, désinstallation propre, et compression LZMA.

4.10 XML pour les paramètres

XML a été choisi plutôt que JSON ou YAML pour sa cohérence sémantique avec le projet (RDF/XML, OWL/XML sont des formats centraux du web sémantique), l'utilisation de la bibliothèque `xml.etree.ElementTree` intégrée à Python, et sa structure hiérarchique adaptée aux groupes de paramètres.

5 Diagramme d'Architecture

```
+-------------------------------------------------------------------+
|                     app.py (Entry Point)                          |
+-----------------------------+-----------------------------------+
                              |
                              v
+-------------------------------------------------------------------+
|                   gui/main_window.py (MainWindow)                 |
|                                                                   |
|  +--------------+ +--------------+ +-------------------------+   |
|  | Graph Viewer | | Query Widget | |    Text KG Widget       |   |
|  | (PyVis +     | | (SPARQL +    | | (Gemini AI + SpaCy NLP) |   |
|  | WebEngine)   | | Highlighter) | |                         |   |
|  +--------------+ +--------------+ +-------------------------+   |
|                                                                   |
|  +----------------+ +--------------+ +----------------------+    |
|  | Ontology Tree  | | Stats Widget | | Dialogs (Namespace,  |    |
|  | (Dock - Left)  | | (Dock-Right) | |  Validation, Infer.) |    |
|  +----------------+ +--------------+ +----------------------+    |
+-----------------------------+-----------------------------------+
                              |
                              v
+-------------------------------------------------------------------+
|                       core/ (Backend)                             |
|                                                                   |
|  +-------------+  +-------------+  +----------------------+     |
|  | RDFManager  |  | SPARQLEngine|  | OntologyManager      |     |
|  | (rdflib)    |--| (rdflib)    |  | (owlready2)          |     |
|  +-------------+  +-------------+  +----------------------+     |
|                                                                   |
|  +-------------+  +-------------+  +----------------------+     |
|  | Reasoner    |  | KnowledgeEx.|  | GeminiExtractor      |     |
|  | (owlready2  |  | (SpaCy NLP) |  | (Gemini API)         |     |
```

```
|   |  HermiT/     |  |              |  |                        |     |
|   |  Pellet)     |  |              |  |                        |     |
|   +--------------+  +--------------+  +------------------------+     |
|                                                                       |
|   +--------------+  +-------------------------------------------+   |
|   | Settings Mgr|  | OntologyValidator (owlready2)             |   |
|   | (XML + .env)|  |                                           |   |
|   +--------------+  +-------------------------------------------+   |
+-----------------------------------------------------------------------+
```

6 Flux de Données

1. **Chargement** : `File` → `RDFManager.load_file()` → Graphe rdflib en mémoire.
2. **Visualisation** : Graphe rdflib → `GraphViewer.display_graph()` → PyVis → HTML → QWebEngineView.
3. **Requête SPARQL** : texte → `SPARQLEngine.execute_query()` → résultats → `QueryWidget.display_results()`.
4. **Extraction NLP** : texte → `KnowledgeExtractor.extract_triples()` → sous-graphe rdflib → fusion dans graphe principal.
5. **Extraction IA** : texte → `GeminiExtractor.extract_triples()` → Turtle → sous-graphe rdflib → fusion.
6. **Raisonnement** : graphe → export OWL → `ReasoningEngine.run_reasoner()` → graphe inféré → diff → fusion.
7. **Ontologie** : graphe → export temp OWL → `OntologyManager.load_ontology()` → hiérarchie → `OntologyTree`.